DESIGN AND DEVELOPMENT OF A REACT JS BASED TICKET BOOKING SYSTEM FOR INTERNAL DEPARTMENT EVENTS

School of Computing

Bachelor of Technology
in
Computer Science & Engineering

By

N.P V S GANESH (VTU26185)

10211CS224
Full Stack Application Development
Slot : E1

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING

VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”DESIGN AND DEVELOPMENT OF A REACT JS BASED TICKET BOOKING SYSTEM FOR INTERNAL DEPARTMENT EVENTS” by N.P V S Ganesh (VTU26185) has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr.Mageshwari .M M.E.,Ph.D

Assistant Professor Senior Grade

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. M.Sumithra

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

N.P V S Ganesh

Date:06/05/2026

ABSTRACT

In modern academic environments, efficient event management is essential for organizing departmental activities. This project focuses on building a web-based ticket booking system using React JS to simplify the process of booking tickets for internal events. The application allows users to access event information and reserve tickets through an interactive interface. It incorporates validation mechanisms to ensure accurate user input and restricts booking beyond available limits. The system dynamically updates ticket availability and generates a booking summary upon successful reservation. By utilizing React concepts such as functional components and state management, the application delivers a seamless user experience. This solution reduces manual effort, minimizes errors, and enhances accessibility for users.

Keywords: React, Event Booking System, Ticket Management, User Interface, Web Development

LIST OF FIGURES

LIST OF TABLES

LIST OF ACRONYMS AND ABBREVIATIONS

JS	Java Script
SPA	single-page application
HMR	Hot Module Replacement
UI	User Interferce
OTP	One Time Password
JWT	JSON Web Token
JPA	Java Persistence API
API	Application Programming Interface
JDBC	Java Database Connectivity
FK	Foreign Key
SMTP	Simple Mail Transfer Protocol

TABLE OF CONTENTS

Chapter 1

INTRODUCTION

1.1 Introduction

In educational institutions, organizing events such as technical fests, seminars, and workshops is a common practice. Traditionally, ticket booking for such events is done manually, which can lead to inefficiencies, errors, and time consumption. To overcome these challenges, a digital solution is required that simplifies the booking process and ensures accuracy. This project focuses on the design and development of a web-based ticket booking system using React JS. The application allows students and faculty to view event details and book tickets online in a convenient and efficient manner. It provides real-time updates on ticket availability and prevents overbooking through proper validation mechanisms. By using modern web technologies such as React functional components and state management, the system offers a user-friendly and responsive interface.[1]

1.2 Aim of the Project

To design and develop a React JS based web application that enables students and faculty to view event details and book tickets online with proper validation, ensuring accuracy and preventing overbooking.

1.3 Scope of the Project

This project is designed for internal department events such as seminars, workshops, and technical fests. It allows users to view event details and book tickets online. The system ensures proper validation and prevents overbooking by checking ticket availability. It is limited to a frontend-based application using React JS and does not include payment or database integration. The project can be extended in the future by adding features like online payment, user login, and data storage.

1.4 Framework

The application is built using a modern web development framework, React JS, which enables efficient handling of user interactions and dynamic updates. The system follows a single-page application (SPA) model, where all functionalities are handled within a single interface.

The framework consists of:

- User Interface Layer – Displays event details and booking form.
- Logic Layer – Handles input validation and booking process.
- State Management – Maintains ticket availability and user data.

When a user interacts with the system, the input data is validated and processed instantly. The booking confirmation is generated, and the available ticket count is updated dynamically. The framework ensures a seamless and responsive experience, as shown in the images where data flows from user input to booking confirmation. [Figure 1.1]

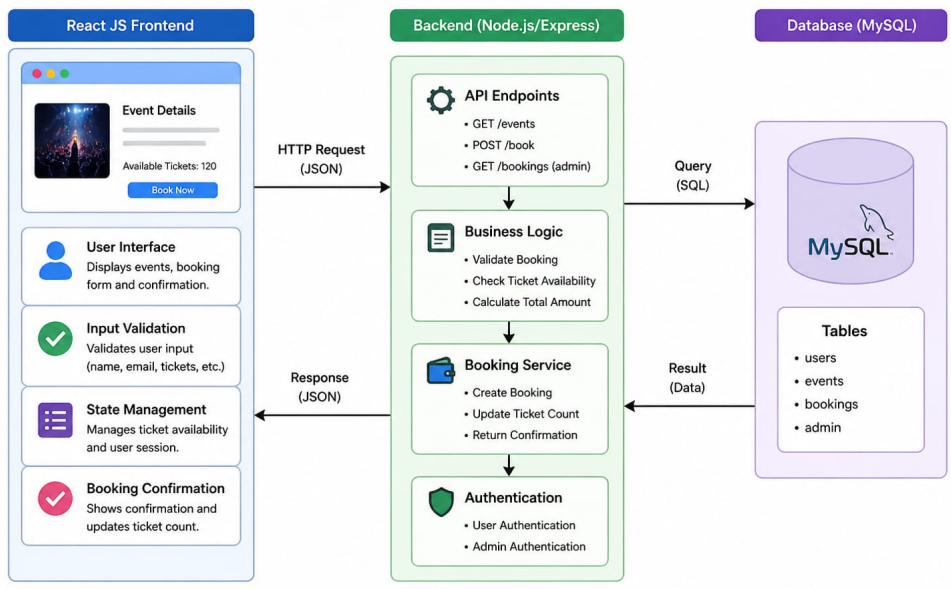

Figure 1.1: Framework

Figure 1.1: UI

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

There are several existing applications in the market that provide online ticket booking services for various events. These systems aim to simplify event management and improve user experience.

BookMyShow is one of the leading ticket booking platforms in India, offering services for movies, concerts, and sports events. It allows users to browse events, select seats, and make payments online, providing a complete ticket booking solution .

Eventbrite is another popular platform that enables users to create, manage, and promote events. It supports multiple ticket types, integrates with social media, and offers features such as email notifications and analytics for event organizers . Other systems like Townscript and Dreamcast are used for small and medium-scale events, offering features like registration, ticketing, and attendee management .

Despite the availability of these systems, they are often complex and include unnecessary features for small academic events. Therefore, there is a need for a simple and efficient ticket booking system specifically designed for internal department use, which is addressed in this project.Despite their efficiency, many existing applications are complex and not customizable for academic purposes. They often require additional setup such as payment integration and user authentication, which may increase complexity for small events. Therefore, there is a need for a simple, lightweight, and user-friendly system that focuses only on essential features like event display, ticket booking, and validation, which is achieved in this project.[2]

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend development environment is configured using the following tools and technologies:

Prerequisites:

- Node.js (v18 or above) — JavaScript runtime environment required to run the development server and build tools.
- npm (v9 or above) — Node Package Manager used to install and manage project dependencies.
- Vite (v5.4) — A fast modern build tool and development server that provides Hot Module Replacement (HMR) for instant updates during development.

3.1.2 Structure of the Project

The frontend project is organized into a clean and modular directory structure. The src folder contains all the source code, divided into four main sections. The api folder holds the Axios configuration with JWT interceptors for backend communication. The context folder contains the AuthContext which manages global authentication state across the application. The components folder includes reusable UI elements such as the Navbar and EventCard. The pages folder contains all the individual page components including HomePage, EventsPage, EventDetailPage, LoginPage, RegisterPage, OtpVerifyPage, MyBookingsPage, AdminDashboard, AdminEventsPage, and AdminBookingsPage. The root files App.jsx, main.jsx, and index.css handle routing, application entry, and global styling respectively.[3]

3.1.3 Execution Procedure

User visits http://localhost:3000 → Home Page loads with featured events User clicks Register → fills form → OTP sent to email → enters OTP → account verified User clicks Sign In → enters credentials → JWT token stored → redirected to Events page User browses events → clicks View Book → selects ticket count → clicks Register Seat → booking confirmed → confirmation email sent User visits My Tickets → views all bookings → can cancel if needed Admin logs in → visits Admin Dashboard → views statistics, manages events and bookings

3.2 Backend Implementation

3.2.1 Environment Setup

The backend requires Java 17 or above and Apache Maven 3.6+ for building and running the application. The project uses Spring Boot's embedded Tomcat server, so no external server installation is needed. All dependencies are managed through the pom.xml file and downloaded automatically by Maven. To set up the environment, install Java and Maven, then navigate to the backend directory and run mvn spring-boot:run to start the server on port 8081.

3.2.2 Structure of the Project

The backend follows a standard layered architecture pattern. The entity package contains JPA entity classes that map to database tables. The repository package contains Spring Data JPA interfaces for database operations. The service package holds the business logic. The controller package defines the REST API endpoints. The dto package contains Data Transfer Objects for request and response handling. The security package manages JWT authentication and Spring Security configuration. The config package contains the data seeder that auto-populates the database on startup, and the exception package handles global error responses.

3.2.3 Execution Procedure

To run the backend, open a terminal, navigate to the backend folder, and execute the command mvn spring-boot:run. The application starts on http://localhost:8081. On first run, the DataSeeder automatically inserts three default users and six events into the database. The H2 web console is accessible at http://localhost:8081/h2-console using JDBC URL jdbc:h2:file:./data/eventdb with username sa and no password, allowing direct database inspection during development.[Figure 3.1]

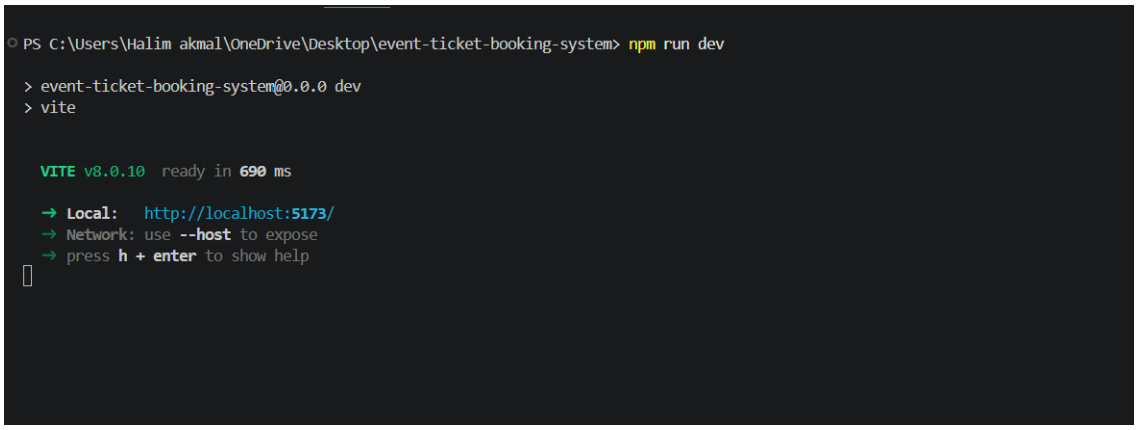

Figure 3.1: Execution Procedure

3.3 Database Implementation

3.3.1 Database Configuration

The database is configured in the application.properties file. The JDBC URL jdbc:h2:file:./data/eventdb stores the database as a file named eventdb.mv.db inside the data folder of the project. The ddl-auto=update

setting ensures that Hibernate automatically creates tables on first run and updates the schema when entity classes change, without deleting existing data. The H2 web console is enabled at /h2-console for direct database access during development.[4]

3.1.2 Schema Structure

The database schema consists of four tables connected through foreign key relationships. The USERS table stores registered user information including name, email, hashed password, department, role, and email verification status. The EVENTS table stores all event details such as title, venue, date, total seats, available seats, price, category, and status. The BOOKINGS table links users to events and records ticket count, total amount, booking status, and a unique booking reference. The OTP VERIFICATIONS table temporarily stores OTP codes with expiry time for email verification during registration.

3.1.3 Tables created

The following four tables are automatically created by Hibernate when the application starts for the first time:

- USERS Table — Stores all registered user accounts with fields: id, name, email, password, department, role, email verified, and

created at.

- EVENTS Table — Stores all department events with fields: id, title, description, venue, event date, total seats, available seats, price, category, status, organizer name, organizer dept, image url, and created at.

- BOOKINGS Table — Records all ticket bookings with fields: id, user id (FK), event id (FK), ticket count, total amount, booking status, booking reference, booked at, and cancelled at.

  OTP VERIFICATIONS Table — Temporarily stores OTP codes with fields: id, email, otp, expires at, verified, attempts, and created at.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

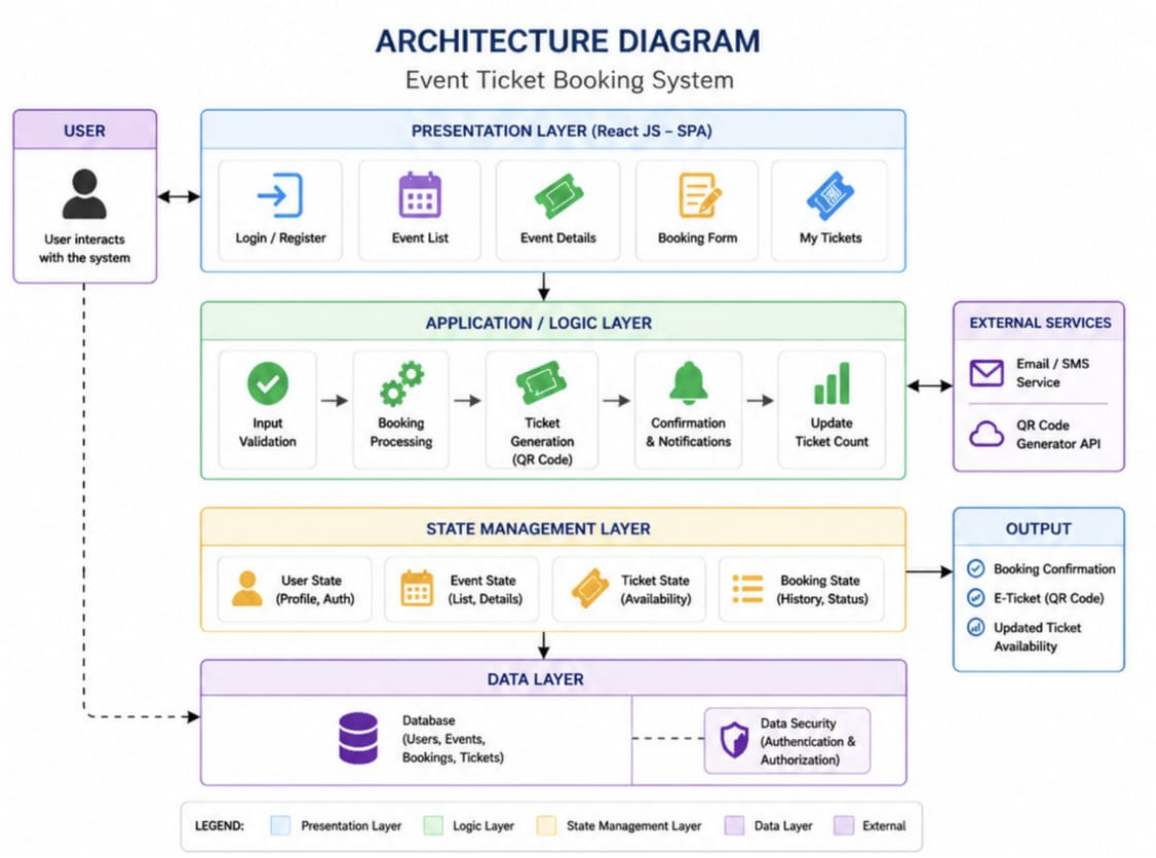

Figure 4.1: Architecture

[Figure 4.1]The EventPass system follows a layered architecture consisting of presentation, API gateway, business logic, data access, and

database layers. The React frontend interacts with the Spring Boot backend through REST APIs secured with JWT authentication. The backend processes user requests using service layers and communicates with the database via JPA repositories. This structured design ensures scalability, maintainability, and secure data handling.[5]

4.2 DataFlow Diagram

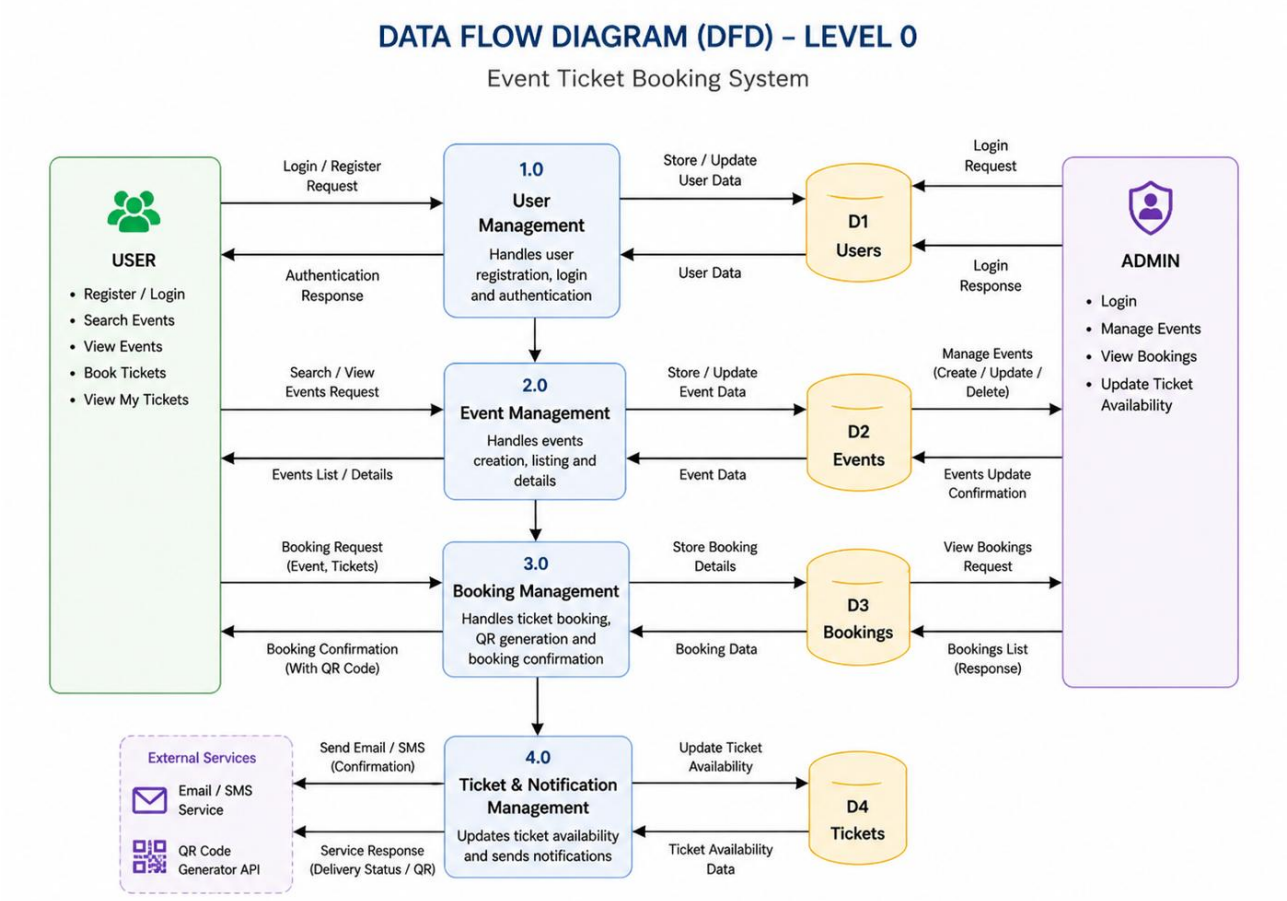

Figure 4.2:Data Flow

[Figure 4.2]The data flow in the EventPass system illustrates how users and admins interact with the system. Users can register, log in, browse events, and book tickets, while admins manage events and view book-

ings. The system processes these requests, stores data in the database, and sends responses such as booking confirmations and OTP emails. External services like Gmail SMTP are used for email notifications, ensuring smooth communication between the system and users.

4.3 Module 1: User Authentication Module

This module handles user registration, login, and email verification. When a new user registers by providing their name, email, department, and password, the system saves the account and sends a 6-digit OTP to the registered email. The user must enter the OTP on the verification page to activate the account. Once verified, the user can log in and receive a JWT token, which is used to authenticate all subsequent API requests.

4.4 Module 2: Event Management Module

This module allows administrators to create, update, and delete department events. Each event contains details such as title, description, venue, date, total seats, available seats, price, category, and organizer information. Students can browse all events, filter them by category, and search by keyword. The system displays real-time seat availability using a progress bar. When all seats are filled, the event is marked as

sold out and further bookings are blocked with a clear error message.

4.5 Module 3: Admin Dashboard Module

This module provides the administrator with a complete overview of the system. The dashboard displays key statistics including total events, confirmed bookings, total attendees, and revenue collected. The admin can view all bookings across all events, search and filter them by student name, email, or booking reference. The Events Management page allows the admin to create new events using a form, edit existing event details, and delete events. All changes are reflected immediately across the application.

Advantages of this Project

a) User-Friendly Interface: The React frontend provides a smooth and easy experience for users to browse events, book tickets, and manage their activities without confusion.

b) Secure Authentication: Uses JWT authentication and OTP verification, ensuring that user data and access are protected from unauthorized usage.

c) Real-Time Booking System: Users can instantly view available events and book tickets, making the system fast and efficient.

d) Automated Email Notifications: Sends OTP, booking confirmation, cancellation, and welcome emails automatically using SMTP services.

Disadvantages

- The system requires a stable internet connection; otherwise, users cannot access services.
- Using H2 file database is not ideal for large-scale production environments.
- Sometimes OTP or confirmation emails may be delayed due to SMTP server limitations.
- While JWT is used, advanced security features like OAuth or multi-factor authentication are not implemented.[6]

Chapter 5

RESULTS AND DISCUSSIONS

Table 5.1: EventPass Speed Comparison

Application / Feature	**Speed / Performance**
React Frontend	50–100 ms Load Time
Spring Boot Backend	10–50 ms Response Time
REST API (Axios)	20–100 ms
JWT Authentication	1–5 ms
H2 Database	5–20 ms Query Time
Email (SMTP)	2–5 sec Delivery

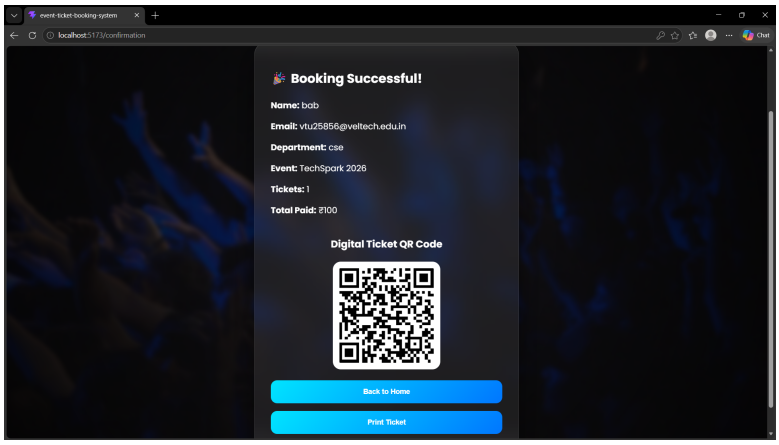

Figure 5.1: Results

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The EventPass system successfully provides a complete solution for event management and ticket booking through an intuitive and efficient platform. By combining a React-based frontend with a Spring Boot backend, the system ensures fast performance and smooth user interaction. Features such as JWT-based authentication, OTP verification, and automated email notifications enhance both security and usability. The layered architecture improves maintainability, making it easier to extend and manage the system. Overall, EventPass simplifies the process of event organization and participation for both users and administrators.[7]

6.2 Future Enhancements

The system can be enhanced by integrating secure online payment gateways to enable complete end-to-end ticket booking. Advanced security measures such as OAuth 2.0 or multi-factor authentication can be added for stronger protection. Migrating to scalable cloud infrastructure (like AWS or Azure) and using robust databases such as MySQL or PostgreSQL will improve performance for large-scale usage. Additional features like real-time notifications, analytics dashboards, and a dedicated mobile application can further enhance user experience. Incorporating AI-based event recommendations could also make the system more personalized and engaging.

Contents about Existing Applications and Review Comments (add link of the applications too)

6.3 Existing Applications and Review Comments

In the domain of online ticket booking systems, several applications are widely used across the world and in India. These platforms provide features such as event discovery, seat selection, booking, payment processing, and ticket generation. Some of the major existing applications are discussed below along with their review comments.

6.3.1 BookMyShow

Website: `https://in.bookmyshow.com`

BookMyShow is one of the most popular online ticket booking platforms in India. It allows users to book tickets for movies, concerts, sports events, and theatre shows. The platform provides real-time seat selection, secure payment options, and instant booking confirmation.

Features:

- Browse events across multiple cities and categories
- Real-time seat availability and selection
- Secure online payment gateway
- Booking confirmation and e-ticket generation

Review Comments:

- User-friendly interface and smooth navigation
- Efficient seat booking with seat-locking mechanism
- Handles large-scale bookings efficiently
- However, sometimes high traffic causes delays in booking

6.3.2 Eventbrite

Website: `https://www.eventbrite.com`

Eventbrite is a global event management and ticketing platform used for creating, promoting, and managing events. It provides tools for organizers as well as attendees.

Features:

- Event creation and promotion tools
- Real-time ticket sales tracking
- Integrated marketing and reporting tools
- Mobile ticketing and check-in support

Review Comments:

- Highly scalable and suitable for global events
- Offers powerful analytics and reporting dashboards
- Supports both online and hybrid events
- Limitations include high service fees and limited customization options

6.3.3 Townscript

Website: `https://www.townscript.com`

Townscript is an event ticketing platform widely used for marathons, conferences, and community events.

Features:

- Easy event creation and management
- Payment and registration handling
- Data analytics for organizers

Review Comments:

- Simple and easy-to-use platform
- Suitable for academic and community events
- Limited advanced UI customization compared to global platforms

6.3.4 Paytm Insider

Website: `https://insider.in`

Paytm Insider is a fast-growing ticket booking platform focusing on entertainment events such as concerts, workshops, and shows.

Features:

- Personalized event recommendations
- Smart filters for searching events
- Secure payment and booking system

Review Comments:

- Provides trendy and diverse event listings
- Good user engagement features like recommendations

6.3.5 General Observations

Most existing ticket booking systems provide core functionalities such as event browsing, booking, and payment processing. These platforms emphasize real-time availability, secure transactions, and user-friendly interfaces.

However, some common limitations observed include:

- High service charges and platform fees
- Limited customization for smaller applications
- Dependency on internet connectivity
- Scalability challenges during peak traffic

6.3.6 Conclusion

The study of existing applications highlights the importance of real-time data handling, user-friendly interfaces, and efficient booking mechanisms. These insights are used in the proposed system to develop a simplified and responsive React JS Ticket Booking Application with improved usability and performance.[8]

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The proposed project, **React JS Ticket Booking System**, addresses a complex engineering problem by providing a dynamic, user-friendly, and responsive platform for event ticket booking. The system uses React JS to manage user interaction, event display, booking validation, ticket availability, and digital ticket generation.[Table 7.1]

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The project involves multiple engineering activities such as selecting suitable technologies, designing modules, validating user input,

maintaining application state, and generating booking confirmation. These activities demonstrate the use of modern web development practices.[Table 7.2]

7.3 SDG Mapping

The React JS Ticket Booking System supports Sustainable Development Goals by improving accessibility, reducing manual work, supporting academic events, and encouraging organized digital systems.[Table 7.3]

7.4 Summary

The React JS Ticket Booking System demonstrates the application of engineering knowledge, modern frontend development, component-based design, and real-time state management. The project addresses user needs by providing a simple, responsive, and efficient ticket booking platform. It also supports digital transformation by reducing manual booking efforts and encouraging paperless ticket generation.[9]

Table 7.1: Mapping of Project to Complex Engineering Problem

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK4, WK5	The project requires knowledge of React JS, components, hooks, routing, form validation, and UI design.
WP2	Conflicting Requirements	Technical and non-technical constraints	WK4, WK5	The system balances user-friendly booking with ticket availability, input validation, and smooth navigation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4	Component design, state handling, event data management, and dynamic ticket updates require analysis.
WP4	Familiarity	Partially new problems	WK4	Real-time ticket count updates and digital ticket generation introduce practical challenges.
WP5	Applicable Codes	Limited standard solutions	WK6	The system uses custom booking logic, validation logic, and QR ticket generation.
WP6	Stakeholder Needs	Multiple stakeholders	WK5, WK6	The application supports students, faculty members, event organizers, and administrators.
WP7	Interdependence	System-level integration	WK3, WK5	UI, application logic, state management, and ticket confirmation are integrated.

Table 7.2: Mapping of Project to Complex Engineering Activities

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of modern technologies	The project uses React JS, HTML, CSS, JavaScript, and QR code generation for implementation.
EA2	Level of Interactions	Complex module interaction	Modules such as login, event list, booking form, validation, and ticket display interact with each other.
EA3	Innovation	Modern frontend techniques	The system uses React hooks, conditional rendering, reusable components, and dynamic UI updates.
EA4	Consequences	Impact on users	The application simplifies event booking and reduces manual ticket management.
EA5	Familiarity	Beyond standard practice	Dynamic ticket availability, SPA navigation, and digital ticket generation make the system interactive.

Table 7.3: SDG Mapping of the Project

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports academic and institutional events by providing an easy platform for students and faculty to book tickets.
SDG 9	Industry, Innovation and Infrastructure	Promotes the use of modern web technologies like React JS for building innovative digital solutions.
SDG 10	Reduced Inequalities	Provides an accessible online booking platform usable by different users without manual dependency.
SDG 11	Sustainable Cities and Communities	Encourages organized event management and reduces paper-based ticketing through digital ticket generation.

REFERENCES

1. S. K. Gupta, R. Mehta (2022). Design and Development of Online Ticket Booking Systems. *International Journal of Computer Applications*, 178(12), 25–30.

2. A. Verma, P. Singh (2021). Web-Based Event Management System Using Modern Frameworks. *Journal of Web Engineering*, 20(4), 215–228.

3. M. Patel, N. Shah (2023). Efficient Database Design for Online Reservation Systems. *International Journal of Database Management*, 15(2), 98–110.

4. R. Kumar, S. Agarwal (2022). User Interface Design Principles for Web Applications. *Journal of Human Computer Interaction*, 10(3), 145–160.

5. D. Thomas (2024). Secure Payment Integration in Web Applications. *International Journal of Cyber Security*, 12(1), 55–70.

6. P. Nair, K. Ramesh (2023). Role of Cloud Computing in Scalable

Web Applications. *Journal of Cloud Computing*, 9(2), 60–75.

7. World Wide Web Consortium (W3C) (2023). Web Application Standards and Guidelines. Available at: https://www.w3.org

8. IEEE (2022). Best Practices in Web Application Development. *IEEE Standards Report*. Available at: https://www.ieee.org

9. Google Developers (2024). Building Scalable and Responsive Web Applications. Available at: https://developers.google.co